

TOPIC 40/60

What is RAG in LLMs?

Retrieval-Augmented Generation.

How dynamic external knowledge bases keep LLMs factual, accurate, and completely up to date.

Ashish Jangra

linkedin.com/in/ashish-jangra



THE PROBLEM

The Isolation of Weights

Once an LLM completes training, its parametric brain is locked in time. It doesn't know anything that happened after its training cutoff date, and it has no access to your private files.

The Pitfalls of "Raw" LLMs

1. **Hallucinations:** Fabricating answers when they lack information.
2. **Cutoff Blindness:** Missing real-time updates.
3. **Zero Context:** No awareness of internal enterprise data.

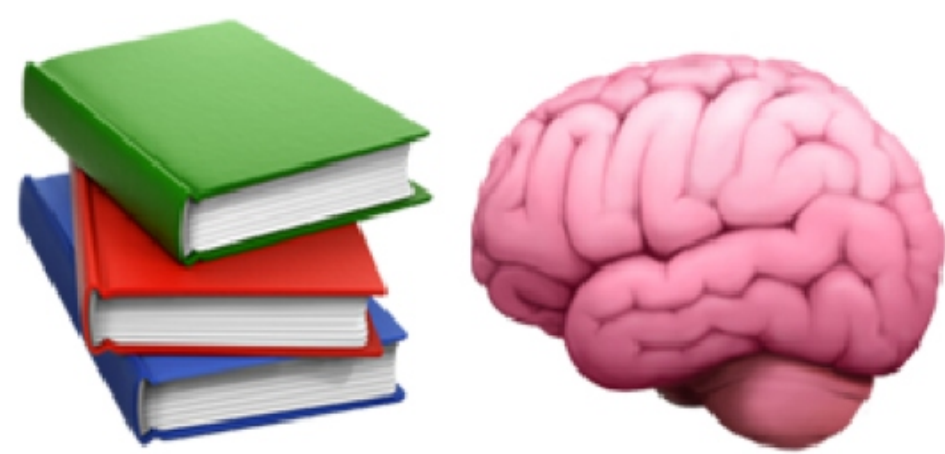
Ashish Jangra

[linkedin.com/in/ashish-jangra](https://www.linkedin.com/in/ashish-jangra)



THE ANALOGY

Open-Book Exams



Closed vs. Open Book

Instead of forcing the LLM to memorize every piece of human knowledge (**Closed-Book**), RAG changes the entire paradigm:

Fine-Tuning (Memorization)

Re-training weights to memorize new textbooks. Hard, expensive, and risks memory decay.

RAG (Open-Book Search)

Handing the LLM a search engine and the exact pages it needs right before it answers.

Ashish Jangra

[linkedin.com/in/ashish-jangra](https://www.linkedin.com/in/ashish-jangra)



ARCHITECTURE

The 3 Pillars of RAG

Every RAG pipeline is built on three fundamental phases operating in harmony:

1. Ingestion (Offline)

Chunking raw files, turning them into vector coordinates, and indexing them in a database.

2. Retrieval (Online)

Analyzing the user query, searching the vector index, and fetching the most relevant passages.

3. Generation (Online)

Feeding the extracted context alongside the original user query to the LLM to produce a highly grounded answer.

Ashish Jangra

[linkedin.com/in/ashish-jangra](https://www.linkedin.com/in/ashish-jangra)



PILLAR 1

Ingestion & Embedding

How we turn messy documents into a clean search-ready vector space:

Step A: Document Chunking

Raw text is split into small, manageable passages (e.g. 500 tokens each) with sliding overlaps to retain semantic continuity.

Step B: Vector Projection

An embedding model converts each chunk into a high-dimensional vector representation—converting meaning into coordinate coordinates.

Step C: Database indexing

These numerical representations are indexed in dedicated Vector DBs (like Pinecone, Milvus, or Qdrant) for lightning-fast lookups.

Ashish Jangra

[linkedin.com/in/ashish-jangra](https://www.linkedin.com/in/ashish-jangra)



PILLAR 2

Semantic Retrieval

Keyword matching (like CTR + F) struggles with concepts.

RAG uses **Semantic Search**:

SIMILARITY METRIC

$$\text{Cosine Similarity}(A, B) = \frac{A \cdot B}{\|A\| \|B\|}$$

By computing the angle between the user's query vector and the document vectors, we extract the top- K chunks that share the closest meaning, regardless of exact keyword matches.

"How do I fix a motor?" matches "diagnosing engine failures" perfectly in vector space.

Ashish Jangra

[linkedin.com/in/ashish-jangra](https://www.linkedin.com/in/ashish-jangra)



PILLAR 3

Context Augmentation

Behind the scenes, your prompt is dynamically reconstructed into a structured, grounded instruction package before reaching the model:

```
// Behind the scenes prompt rewrite
```

You are an expert assistant. Answer the user prompt based ONLY on the reliable context below:

[CONTEXT]:

"Section 4.1: The primary bypass valve must be rotated clockwise to disengage pressure thresholds during manual overriding."

[USER QUERY]: How do I override the pressure system?

Ashish Jangra

[linkedin.com/in/ashish-jangra](https://www.linkedin.com/in/ashish-jangra)



STRATEGIES

RAG vs. Fine-Tuning

Dimension	RAG	Fine-Tuning
Real-time Data	Instant updates	Requires retune
Hallucination Control	High (linked facts)	Low to Medium
New Skills	Poor (uses base)	Excellent (teaches style)
Implementation Cost	Low setup	High compute & math

Rule of thumb: Use **RAG** for factual, fast-changing documents. Use **Fine-Tuning** to adapt the tone, formatting, structure, or specialized vocabulary.



ADVANCED TACTICS

Beyond Simple RAG

Standard RAG struggles with complex queries. Production-grade pipelines apply advanced rerouting techniques:

1. Query Rewriting & Expansion

Using a small LLM to rewrite vague queries to better capture relevant vectors in the database.

2. Two-Stage Retrieval (Re-ranking)

Retrieving 50 chunks quickly with dense vector models, then using a slow but precise Cross-Encoder model to pick the absolute top 5 chunks.

3. Parent-Child Chunking

Searching small, highly specific sub-chunks to locate data, but returning the larger parent document context to the LLM to preserve complete context.

Ashish Jangra

[linkedin.com/in/ashish-jangra](https://www.linkedin.com/in/ashish-jangra)



RAG

CONCEPT UNLOCKED



Turns inference into an "open-book exam"



Indexes text chunks using dense semantic vectors



Controls hallucinations without retraining costs



Optimized by Re-ranking & Parent-Child setups

CONNECT WITH ME



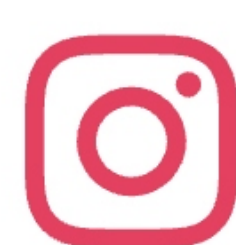
LinkedIn



GitHub



Kaggle



Instagram

Ashish Jangra

linkedin.com/in/ashish-jangra

